

Guide de Sécurité & Migration Auth — La Termitière

Version : 1.0

Statut : Production

Cible : Équipe technique / DevOps

Dernière mise à jour : Juillet 2026

1. Objectif

Ce document centralise les pratiques de sécurité pour la base de données PostgreSQL (gérée par Supabase) et la procédure de migration de l'authentification.

L'objectif principal est de verrouiller la base de données pour empêcher tout accès public (anonyme).

2. La Sécurité des Données (Row Level Security - RLS)

Par défaut, une base de données peut être exposée si elle n'est pas protégée. Dans La Termitière, nous utilisons la fonctionnalité RLS (Row Level Security) de PostgreSQL pour bloquer les requêtes non autorisées au niveau même de la base de données.

2.1 Le script de verrouillage (`secure-auth.sql`)

Ce script SQL est vital. Il parcourt toutes les tables métier (qui commencent par `tp_`) et leur applique un bouclier strict :

- Révocation totale des droits pour l'utilisateur public (`anon`).
- Seuls les utilisateurs authentifiés par Supabase Auth (`authenticated`) peuvent lire ou écrire.

Extrait de la logique appliquée :

```
-- Fermer toutes les tables tp_* : accès réservé aux comptes authentifiés
execute format('alter table public.%I enable row level security;', t);
execute format('drop policy if exists %I on public.%I;', t||'_test_all', t);
execute format('create policy %I on public.%I for all to authenticated using (true) with
execute format('revoke all on public.%I from anon;', t);
execute format('grant all on public.%I to authenticated;', t);
```

Conclusion : Si un attaquant tente de requêter l'API REST (PostgREST) sans token JWT valide, la base de données renverra une erreur d'autorisation.

3. Migration des Utilisateurs vers Supabase Auth

Lors du passage à Supabase, les anciens utilisateurs (stockés dans la table `tp_users`) doivent être convertis en véritables comptes Auth sécurisés. C'est le rôle du script `auth-migrate.mjs`.

3.1 Fonctionnement du script

- Le script nécessite la clé secrète **Service Role** (`SUPABASE_SERVICE_KEY`), car la création de compte via l'API Admin outrepassse les règles de sécurité classiques.
- Il lit la table `tp_users`.
- Il génère un e-mail synthétique pour chaque utilisateur (car Supabase Auth est basé sur l'e-mail par défaut). Format : `login@latermitiere.local`.
- Il génère un mot de passe temporaire : si le login fait plus de 6 caractères, le mot de passe est le login. Sinon, il ajoute le suffixe `-2026` (ex: `admin-2026`).
- Il crée le compte via l'API d'administration et insère les métadonnées dans la table de liaison `profiles`.

3.2 Exécution de la migration

Puisque Node.js 20 est installé nativement sur le VPS, tu peux lancer le script directement :

```
# 1. Se placer dans le dossier contenant le script
cd /home/bawa/termitiere-platform/migration/supabase

# 2. Installer les dépendances (si pas déjà fait)
npm install

# 3. Configuration des variables d'environnement requises
export SUPABASE_URL="https://api.latermitiere.com"
# (Tu trouveras cette clé dans le fichier /home/bawa/supabase/docker/.env)
export SUPABASE_SERVICE_KEY="TA_VRAIE_CLE_SERVICE_ROLE"

# 4. Exécution du script
# (Le bypass NODE_TLS_REJECT_UNAUTHORIZED=0 est normal en local pour l'administration)
NODE_TLS_REJECT_UNAUTHORIZED=0 node auth-migrate.mjs
```

4. Configuration du Pare-feu (UFW)

4.1 Installation (UFW n'est pas préinstallé sur Debian minimal)

Les images Debian fournies par les hébergeurs VPS sont souvent "minimales" : UFW n'est pas inclus par défaut. Voici comment l'installer et le configurer :

```
# 1. Installer UFW
sudo apt update && sudo apt install ufw -y

# 2. 🚨 VITAL : Autoriser SSH en PREMIER (sinon tu te coupes l'accès !)
sudo ufw allow ssh

# 3. Autoriser le trafic web public (pour Caddy)
sudo ufw allow http
sudo ufw allow https

# 4. Activer le pare-feu
sudo ufw enable
```

(Réponds `y` quand UFW avertit que ça peut couper les connexions. Tu as autorisé SSH juste avant, donc c'est sécurisé).

Vérification :

```
sudo ufw status
```

4.2 Le Piège Docker + UFW (Critique à comprendre)

🚨 UFW seul ne suffit pas à protéger les ports Docker !

Docker modifie directement les règles `iptables` (le pare-feu bas niveau de Linux) avec une priorité supérieure à UFW. Conséquence : même si tu fais `ufw deny 5432`, Docker peut quand même exposer ce port à internet.

La vraie solution : binder les ports sensibles sur `127.0.0.1` dans Docker.

Dans le fichier `/home/bawa/supabase/docker/docker-compose.yml`, les ports PostgreSQL sont configurés ainsi :

```
# ✅ CORRECT (accessible uniquement en local) :
- 127.0.0.1:${POSTGRES_PORT}:5432
- 127.0.0.1:${POOLER_PROXY_PORT_TRANSACTION}:6543

# ❌ DANGEREUX (accessible depuis internet entier) :
- ${POSTGRES_PORT}:5432
```

Cette configuration a déjà été appliquée sur ce VPS. Pour vérifier :

```
docker compose ps supavisor
# Tu dois voir : 127.0.0.1:5432->5432/tcp (et non 0.0.0.0:5432)
```

5. Bonnes Pratiques de Sécurité Quotidiennes

- Ne jamais commiter de fichier `.env`.
- Ne jamais partager la `SERVICE_ROLE_KEY`. Elle donne un accès "Dieu" (bypass RLS) à toute la base de données.
- Le port `5432` (PostgreSQL) ne doit jamais être accessible depuis internet. Double protection : UFW + bind sur `127.0.0.1` dans Docker (voir Section 4.2).
- Les tokens générés pour le CI/CD (GitHub PAT) doivent avoir une durée de vie limitée (90 jours max) avec les permissions strictes minimales (`read:packages` uniquement).